

Project Documentation

An offline-first campus intelligence platform for skill exchange, peer learning, and balanced project teams.

Frontend Next.js, TypeScript, Tailwind	Backend FastAPI, SQLAlchemy	Intelligence Python provider facade	Data SQLite or PostgreSQL
--	--	--	--

Submission summary

- Connect students through complementary teach and learn skills, learning goals, availability, and project interests.
- Turn natural-language profiles and project briefs into structured skills and actionable recommendations.
- Provide a self-contained local path using SQLite, mock or local AI providers, and offline-first client behavior.

Prepared for hackathon evaluation | Source, technical references, diagrams, and interface previews are included in this submission package.

The Challenge and Solution

Campus talent is often hidden in disconnected profiles, chat groups, and informal networks. AI Skill Exchange makes complementary skills, peer teaching, and project opportunities visible in one student-centered workspace.

Discover Students describe what they can teach, what they want to learn, and where they can contribute.	Reason The intelligence layer extracts skills, applies taxonomies and guardrails, and scores complementary fit.	Activate The product supports peer sessions, a credit ledger, project teams, and campus-level insight.
---	---	--

What evaluators can explore

Capability	Evaluator outcome
Skill analysis	Natural language becomes structured, reviewable skill information.
Peer matching	Learners see complementary peers and a clear match rationale.
Team builder	Project briefs produce balanced candidate team compositions.
Credit economy	Teaching and learning participation can be recognized transparently.
Campus insights	Aggregate skill gaps can guide student and faculty action.

The detailed demo sequence is available in *docs/source-documentation/DEMO_GUIDE.md* inside the submission package.

Architecture

The system separates the presentation, API/domain, intelligence, and persistence responsibilities. Provider boundaries keep the local-first default deployable without a required cloud dependency, while allowing production substitutions.

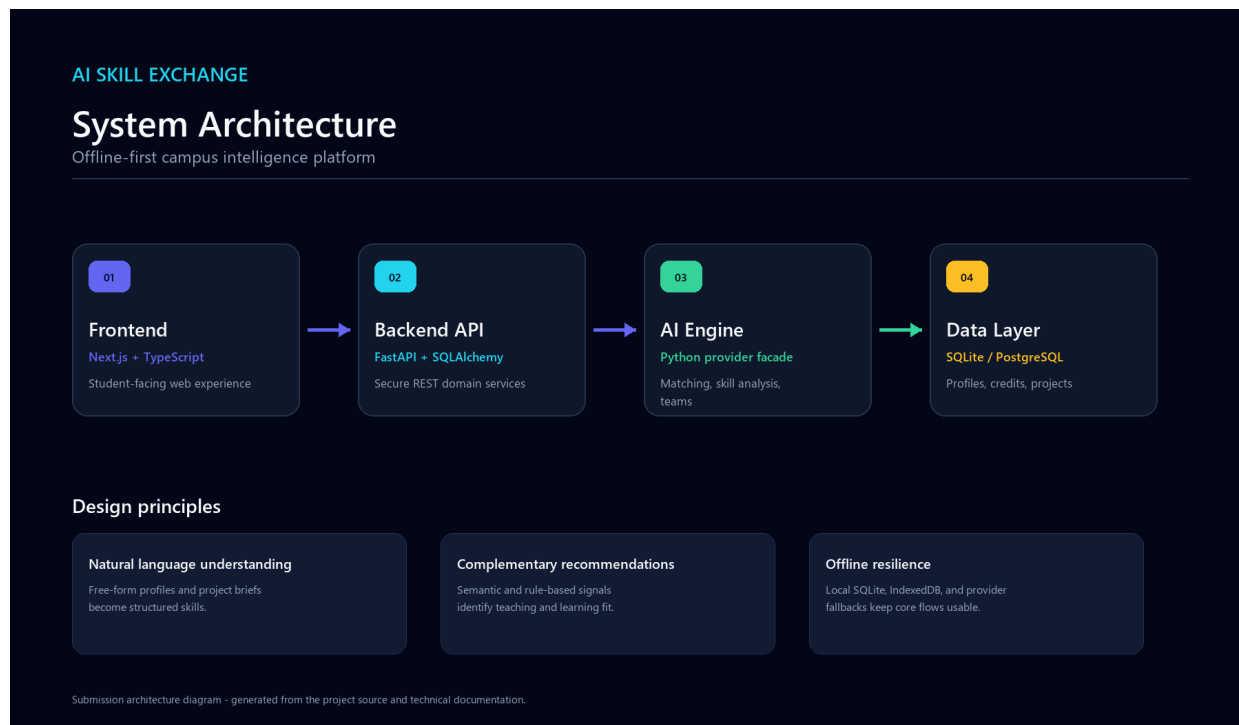


Figure 1. Layered architecture: Next.js frontend, FastAPI services, Python intelligence, and SQLite or PostgreSQL persistence.

Architecture choices

- A provider facade supports deterministic mock, local AI, and online AI options behind shared interfaces.
- SQLAlchemy and configuration keep SQLite development and PostgreSQL production deployment paths aligned.
- The frontend uses client-side offline storage and a synchronization workflow to preserve interaction resilience.
- Authentication, ownership checks, input validation, rate limits, and AI guardrails are part of the security design.

Intelligence and Data Flow

The product uses structured rules and optional semantic capabilities to turn a student's stated intent into useful, accountable recommendations. The same domain data supports peer exchange and campus-level insights.

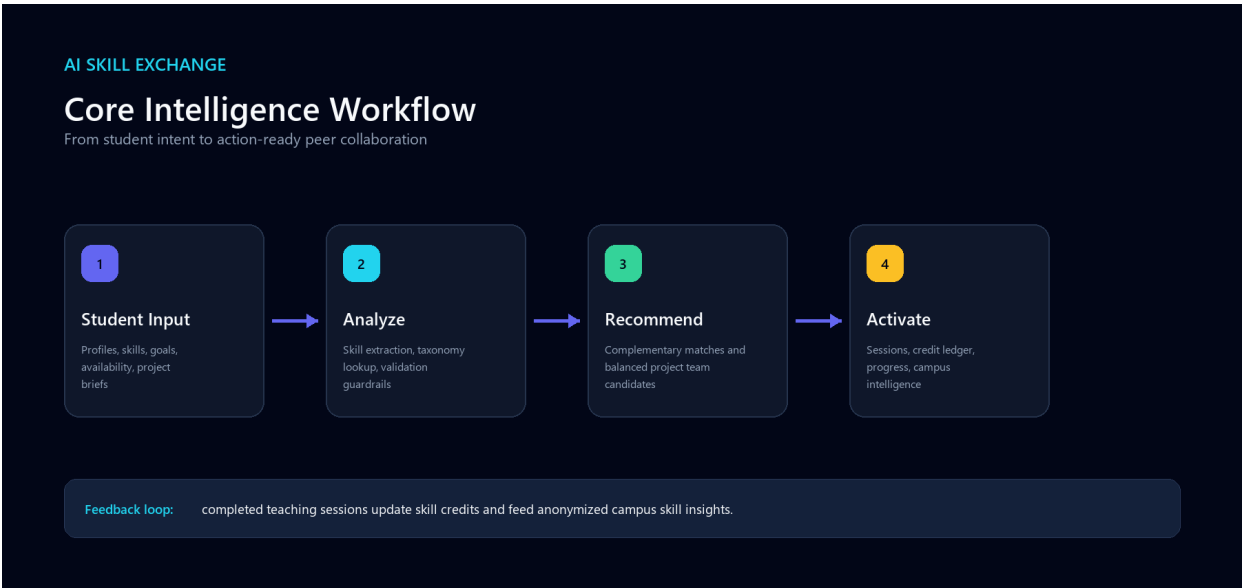


Figure 2. Core intelligence workflow from submitted information to peer and project outcomes.

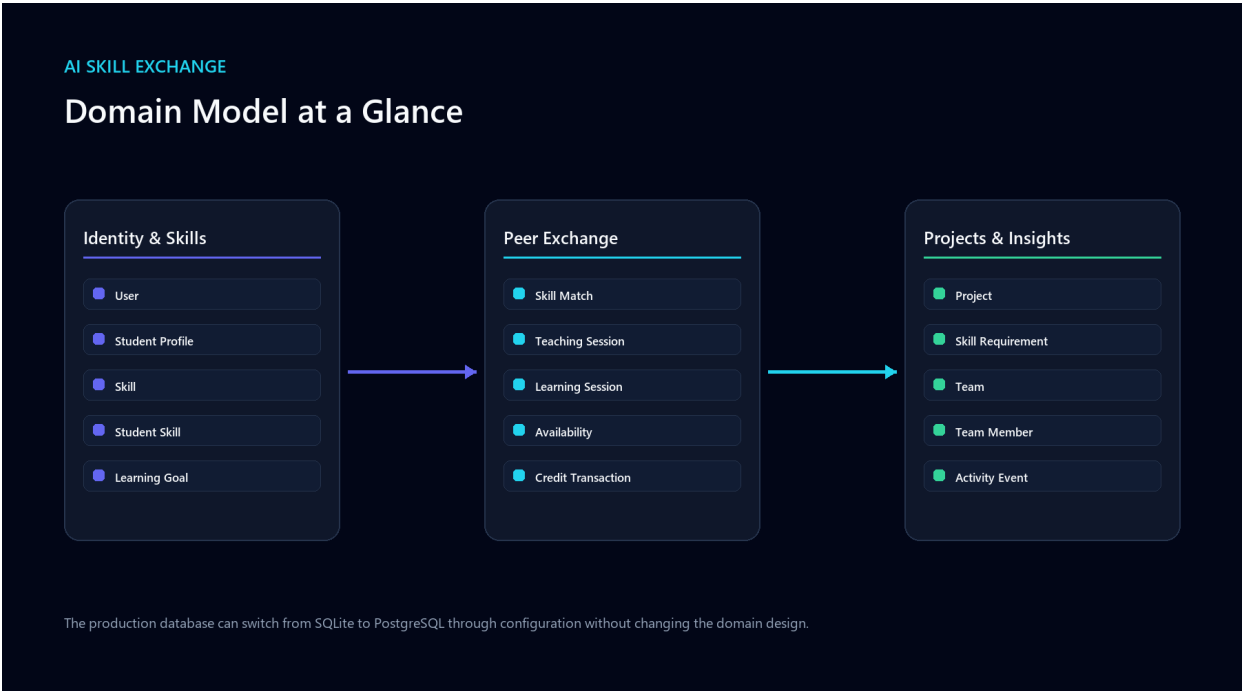


Figure 3. Key domain entities organized around skills, peer exchange, projects, and campus intelligence.

Implementation and Evaluation

The complete implementation is contained in *src/project-source-code/*. It includes the frontend, backend, AI modules, database seeds and migrations, automated tests, Docker configuration, and original project documentation.

Area	Included implementation
Web client	Next.js application with Dashboard, Skills, Matches, Sessions, Credits, Projects, Team Builder, Campus Insights, and offline sync views.
API	FastAPI routes covering authentication, profiles, skills, goals, matching, projects, teams, sessions, credits, activities, and campus intelligence.
AI modules	Skill analysis, matching, project analysis, team formation, guardrails, embeddings, and replaceable providers.
Quality	Backend and AI tests, migration files, API verification scripts, environment template, and development/deployment guides.

Run locally

- Copy ``src/project-source-code/.env.example`` to ``.env`` and configure local values as needed.
- Install backend dependencies from ``src/project-source-code/backend/requirements.txt``, then run FastAPI on port 8000.
- Install frontend dependencies from ``src/project-source-code/frontend/package.json``, then start Next.js on port 3000.
- Use the seeded personas and the 3-minute demo mode to review matching, team-building, and credit flows quickly.

Submission Guide

The archive is clean for evaluation. It excludes generated dependencies, build output, virtual environments, and caches while retaining the project's source, tests, configuration, and documentation.

Path	Purpose
README.md	Submission overview and concise local-start instructions.
src/project-source-code/	Full original implementation and all source project materials.
docs/	This PDF, architecture visual, supporting diagrams, and selected original documentation.
screenshots/	Static visual previews of the Dashboard and Team Builder interfaces.
data/	Notes for the local SQLite demonstration data and reproducible seeds.
requirements.txt	Convenient root entry point for backend and AI dependencies.

Evaluation path

- Read this document and `README.md` for the intent, technical boundaries, and start instructions.
- Open `docs/architecture.png`, `docs/other-diagrams/`, and `screenshots/` for a fast visual tour.
- Review `src/project-source-code/DEMO\_GUIDE.md` for the recommended three-minute product demonstration.
- Inspect tests and API routes in `src/project-source-code/tests/` and `src/project-source-code/backend/app/` for implementation evidence.

AI Skill Exchange is designed to make campus collaboration more discoverable, practical, and resilient.